Esercizi su Stream API

Esercizio - Divisione

- Dividi ogni numero di una lista di interi per 2 (ignorando il resto) e raccogli in un set.
- Applica a

```
List<Integer> numbers = asList(1, 2, 3, 4, 5);
```

Esercizio - Divisione - Soluzione

```
import java.util.*;
import java.util.stream.Collectors;
import static java.util.Arrays.asList;


public class Esercizio2 {
    public static void main(String[] args){
        List<Integer> numbers = asList(1, 2, 3, 4, 5);


        Set<Integer> risultato = exercise_2(numbers);
        System.out.println(risultato);
    }


    public static Set<Integer> exercise_2(List<Integer> numbers) {
        return numbers.stream().map(n -> n / 2).collect(Collectors.toSet());
    }

}
```

Esercizio - Stringa più lunga

- Trova la lunghezza della stringa più lunga
- Applica a

```
List<String> strings = asList("Lorem", "ipsum", "dolor", "sit", "amet,",
"consectetur", "adipiscing", "elit.");
```

Esercizio - Stringa più lunga - Soluzione

```
import java.util.*;
import static java.util.Arrays.asList;


public class Esercizio3 {
    public static void main(String[] args){
        List<String> strings = asList("Lorem", "ipsum", "dolor", "sit", "amet,", "consectetur", "adipiscing", "elit.");

        Optional<Integer> risultato = exercise_3(strings);
        System.out.println(risultato.orElse(0));
    }


    public static Optional<Integer> exercise_3(List<String> strings){
        return strings.stream().map(String::length).reduce(Math::max);
    }
}
```

Esercizio stringhe

- Dato il seguente array di stringhe:

```
String array[] = {“Mela”, “Pera”, “Arancia”, “Fragola”, “Kiwi”, “Arancia”};
```

- si scriva uno stream che stampa a video l’insieme delle stringhe diverse. L’output a video dovrà essere il seguente

Mela

Pera

Arancia

Fragola

Kiwi

Esercizio stringhe - Soluzione

```
import java.util.Arrays;

public class Esercizio3 {
  public static void main(String args[]) {

    String array[] = {"Mela", "Pera", "Arancia", "Fragola", "Kiwi", "Arancia"};
    Arrays.stream(array).distinct().forEach(s->
      System.out.println(s));
  }
}
```

Esercizio numeri

- Data la seguente lista di numeri interi

```
List<Integer> numeri = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

- Si scriva uno stream che permette di selezionare il primo numero pari.

Esercizio numeri - Soluzione

```
import java.util.Arrays;
import java.util.List;
import java.util.Optional;
public class Esercizio5 {
  public static void main(String[] args) {
    List<Integer> numeri = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
    Optional<Integer> risultato = numeri.stream()
        .filter(n -> n % 2 == 0)
        .findFirst();
    if (risultato.isPresent()) {
      System.out.println("Il primo numero pari è: " + risultato.get());
    }
  }
}
```

Esercizio pari

- Data la lista di interi definita nell'Esercizio 5, si scriva uno stream che permetta di contare il numero di elementi pari.

Esercizio pari - Soluzione

```
import java.util.Arrays;
import java.util.List;

public class Esercizio6 {
  public static void main(String[] args) {
    List<Integer> numeri = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

    long risultato = numeri.stream()
        .filter(n -> n % 2 == 0)
        .count();

    System.out.println("Ci sono " + risultato + " numeri pari");
  }
}
```

Esercizio - Primi 5 numeri divisibili per 17

- Trova i primi 5 numeri divisibili per 17 generati da un flusso infinito
- Applica a

```
IntStream numbers = new Random(0).ints().map(Math::abs);
```

Esercizio - Primi 5 numeri divisibili per 17 - Soluzione

```
import java.util.*;
import java.util.stream.*;

public class Esercizio4 {
   public static void main(String[] args){
      IntStream numbers = new Random().ints().map(Math::abs);
      List<Integer> risultato = exercise_4(numbers);
      System.out.println(risultato);
   }
   public static List<Integer> exercise_4(IntStream numbers){
      return numbers.filter(n -> n % 17 == 0).limit(5).mapToObj(n -> n).collect(Collectors.toList());
   }
}
```

Esercizio maiuscole

- Data la seguente lista di stringhe

```
List<String> parole = Arrays.asList(“luna”, “stitch”, “miguel”,
 “speedy”);
```

- Si crei una nuova lista con le parole della lista parole con tutte le lettere maiuscole.
- Si utilizzi la notazione ::

Esercizio maiuscole - Soluzione

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class Esercizio7 {
  public static void main(String[] args) {
    List<String> parole = Arrays.asList("luna", "stitch", "miguel", "speedy");
    List<String> paroleMaiuscole =
        parole.stream()
          .map(String::toUpperCase)
          .collect(Collectors.toList());
    System.out.println("Parole maiuscole: " + paroleMaiuscole);
  }
}
```

Esercizio spesa

- Data la classe Spesa

```
public class Spesa {
 //L'importo in euro
 private double importo;
 //La tipologia di spesa
 private String tipologia;

 public Spesa(double importo, String tipologia) {
  this.importo = importo;
  this.tipologia = tipologia;
 }
 public double getImporto() {
  return importo;
 }
 public String getTipologia() {
  return tipologia;
 }
 public String toString() {
  return "Spesa [importo=" + importo + ", tipologia=" + tipologia + "]";
 }
}
```

Esercizio spesa - soluzione

- Ottenere lo stesso risultato del seguente codice non avvalendosi degli stream.

```
Map<String, List<Spesa>> mappa =
spese.stream()
  .collect(Collectors.groupingBy(spesa ->
   spesa.getTipologia()));
```

Soluzione

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;
public class Esercizio4 {
  public static void main(String[] args) {
    List<Spesa> spese = new ArrayList<Spesa>();
    spese.add(new Spesa(10.00, "vitto"));
    spese.add(new Spesa(15.50, "cinema"));
    spese.add(new Spesa(300.00, "alloggio"));
    spese.add(new Spesa(32.70, "vitto"));
    spese.add(new Spesa(52.10, "vitto"));
    // come nel testo
    Map<String, List<Spesa>> mappa2 = spese.stream().collect(Collectors.groupingBy(spesa -> spesa.getTipologia()));
    System.out.println(mappa2.keySet());
    System.out.println(mappa2.get("vitto"));
    System.out.println(mappa2.get("cinema"));
    System.out.println(mappa2.get("alloggio"));
    Map<String, List<Spesa>> mappa = new HashMap<String, List<Spesa>>();
    for (Spesa spesa : spese) {
      String tipologia = spesa.getTipologia();
      if (mappa.containsKey(tipologia)) {
        mappa.get(tipologia).add(spesa);
      } else {
        List<Spesa> nuoveSpese = new ArrayList<>();
        nuoveSpese.add(spesa);
        mappa.put(tipologia, nuoveSpese);
      }
    }
    System.out.println(mappa.keySet());
    System.out.println(mappa.get("vitto"));
    System.out.println(mappa.get("cinema"));
    System.out.println(mappa.get("alloggio"));
  }
}
```

Esercizio conto

- Si aggiunga il metodo toString alla classe Conto

```
public class Conto {
  private int saldo;
  public Conto() {
    this(0);
  }
  public Conto(int ammontare) {
    ingresso(ammontare);
  }
  public void ingresso(int ammontare) {
    saldo+=ammontare;
  }
  public void uscita(int ammontare) {
    saldo-=ammontare;
  }
  public int getSaldo() {
    return saldo;
  }
}
```

Esercizio conto

- Si definisca un metodo statico in una classe denominata ContoDemo con nome selezionaContiSaldoMaggioreDi che, dati una lista di conti e una soglia, restituisce la lista di conti con saldi superiori alla soglia specificata. Si utilizzino gli stream e le espressioni lambda. Si provi il codice con le seguenti istruzioni inserite in un metodo main:

```
List<Conto> conti = new ArrayList<Conto>();
conti.add(new Conto(100));
conti.add(new Conto(10));
conti.add(new Conto(50));
conti.add(new Conto(200));
conti.add(new Conto(250));

List<Conto> contiSaldoMaggiore50 =
ContoDemo.selezionaContiSaldoMaggioreDi(conti, 50);

System.out.println(contiSaldoMaggiore50);
```

Esercizio conto - Soluzione

```
public class Conto {
  private int saldo;
  public Conto() {
    this(0);
  }
  public Conto(int ammontare) {
    ingresso(ammontare);
  }
  public void ingresso(int ammontare) {
    saldo+=ammontare;
  }
  public void uscita(int ammontare) {
    saldo-=ammontare;
  }
  public int getSaldo() {
    return saldo;
  }
  public String toString() {
    return ""+saldo;
  }
}
```

Esercizio conto - Soluzione

```
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;
public class ContoDemo {
  public static List<Conto> selezionaContiSaldoMaggioreDi(List<Conto> conti, int saldo) {
    return conti.stream()
        .filter(c -> c.getSaldo() > saldo)
        .collect(Collectors.toList());
  }
  public static void main(String[] args) {
    List<Conto> conti = new ArrayList<Conto>();
    conti.add(new Conto(100));
    conti.add(new Conto(10));
    conti.add(new Conto(50));
    conti.add(new Conto(200));
    conti.add(new Conto(250));
    List<Conto> contiSaldoMaggiore50 =
       ContoDemo2.selezionaContiSaldoMaggioreDi(conti, 50);
    System.out.println(contiSaldoMaggiore50);

  }
}
```

Esercizio conto 2

- Date le classi Conto e ContoDemo definite nell'Esercizio 1, definire un ulteriore metodo statico nella classe ContoDemo con nome verificaSaldoMaggioreDi che, dati una lista di conti e una soglia, verifica se tutti i conti hanno saldo maggiore della soglia specifica. Si utilizzino gli stream e le espressioni lambda. Si provi il codice con le seguenti istruzioni inserite in un metodo main:

```
ListConto conti = new ArrayListConto();
conti.add(new Conto(100));
conti.add(new Conto(10));
conti.add(new Conto(50));
conti.add(new Conto(200));
conti.add(new Conto(250));

System.out.println(ContoDemo.verificaSaldoMaggioreDi(conti, 50));
```

Esercizio conto 2 - Soluzione

```
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;
public class ContoDemo {
 public static List<Conto> selezionaContiSaldoMaggioreDi(List<Conto> conti, int saldo) {
  return conti.stream()
      .filter(c -> c.getSaldo() > saldo)
      .collect(Collectors.toList());
 }
 public static boolean verificaSaldoMaggioreDi(List<Conto> conti, int saldo) {
  return conti.stream().allMatch(c -> c.getSaldo() > saldo);
 }
 public static void main(String[] args) {
  List<Conto> conti = new ArrayList<Conto>();
  conti.add(new Conto(100));
  conti.add(new Conto(10));
  conti.add(new Conto(50));
  conti.add(new Conto(200));
  conti.add(new Conto(250));
  List<Conto> contiSaldoMaggiore50 =
     ContoDemo2.selezionaContiSaldoMaggioreDi(conti, 50);
  System.out.println(contiSaldoMaggiore50);

  System.out.println(ContoDemo2.verificaSaldoMaggioreDi(conti, 50));

 }
}
```

Esercizio - Persone

- Hai una lista di persone, ciascuna rappresentata da un oggetto Persona con i seguenti attributi: nome, cognome, età e una lista di String rappresentanti le loro passioni.
- Utilizza la Stream API per:

1. **Filtrare** la lista e ottenere solo le persone maggiorenni.
2. **Ordinare** le persone filtrate in base all'età in modo crescente.
3. **Raggruppare** le persone per passione, creando una mappa dove la chiave è la passione e il valore è una lista di persone che condividono quella passione come prima nella loro lista.
4. **Stampare** per ogni passione, il numero di persone interessate e la lista dei loro nomi e cognomi.

Esercizio - Persone

```
import java.util.*;
public class Persona {
   private String nome, cognome;
   private int eta;
   private List<String> passioni;
   public Persona(String nome, String cognome, int eta, List<String> passioni){
      this.nome = nome;
      this.cognome = cognome;
      this.eta = eta;
      this.passioni = passioni;
   }
   public String getNome(){
      return nome;
   }
   public String getCognome(){
      return cognome;
   }
   public int getEta(){
      return eta;
   }
   public List<String> getPassioni(){
      return passioni;
   }
   public String toString(){
      return nome + " " + cognome;
   }
}
```

Esercizio - Persone - Soluzione

```
public class EsercizioStream {
    public static void main(String[] args) {
        // Crea una lista di persone (esempio)
        List<Persona> persone = Arrays.asList(
            new Persona("Mario", "Rossi", 30, Arrays.asList("calcio", "lettura")),
            // ... altre persone
        );
        // Soluzione
        Map<String, List<Persona>> personePerPassione = persone.stream()
            .filter(p -> p.getEta() >= 18) // Filtra i maggiorenni
            .sorted(Comparator.comparing(Persona::getEta)) // Ordina per età
            .collect(Collectors.groupingBy(p -> p.getPassioni().get(0))); // Raggruppa per la prima passione (puoi modificare)
        // Stampa i risultati
        personePerPassione.forEach((passione, personeConPassione) -> {
            System.out.println("Passione: " + passione);
            System.out.println("Numero persone: " + personeConPassione.size());
            personeConPassione.forEach(p -> System.out.println("  " + p.nome + " " + p.cognome));
        });
    }
}
```

Esercizio - Persone - Sfide aggiuntive

- **Calcolo statistiche:** Calcola l'età media delle persone per ogni passione.
- **Personalizzazione della stampa:** Formatta l'output in modo più elegante.

Esercizio - Transazioni

- **Scenario:** Hai una lista di transazioni bancarie, rappresentate da una classe Transazione con attributi data, importo e tipo (ad esempio, "credito" o "debito").
- Utilizza la Stream API per:

1. **Filtrare** le transazioni effettuate in un determinato mese dell'anno.
2. **Calcolare** l'importo totale dei crediti e dei debiti separatamente.
3. **Trovare** la transazione con l'importo più alto in valore assoluto (sia per i crediti che per i debiti).
4. **Ordinare** le transazioni in base alla data in ordine cronologico decrescente.

Esercizio - Transazioni

```
import java.time.LocalDate;
public class Transazione {
   private LocalDate data;
   private double importo;
   private String tipo;
   public Transazione(LocalDate data, double importo, String tipo){
      this.data = data;
      this.importo = importo;
      this.tipo = tipo;
   }
   public LocalDate getData(){
      return data;
   }
   public double getImporto(){
      return importo;
   }
   public String getTipo(){
      return tipo;
   }
   public String toString(){
      return data + " , " + importo + " , " + tipo;
   }
}
```

Esercizio – Transazioni - Soluzione

```
import java.util.*;
import java.util.stream.Collectors;
import java.time.*;

public class Esercizio6 {
   public static void main(String[] args){
      List<Transazione> transazioni = Arrays.asList(
         new Transazione(LocalDate.of(2023, Month.NOVEMBER, 15), 100.0, "credito"),
         new Transazione(LocalDate.of(2023, Month.APRIL, 11), 200.0, "debito"),
         new Transazione(LocalDate.of(2023, Month.AUGUST, 9), 300.0, "credito"),
         new Transazione(LocalDate.of(2023, Month.OCTOBER, 2), 400.0, "debito"),
         new Transazione(LocalDate.of(2023, Month.NOVEMBER, 8), 500.0, "credito")
      );

      // Punto 1: filtrare le transazioni effettuate in un determinato mese dell'anno
      List<Transazione> transazioniNovembre = transazioni.stream().filter(t -> t.getData().getMonth() == Month.NOVEMBER).collect(Collectors.toList());
      System.out.println(transazioniNovembre);


      // Punto 2: Calcolare l'importo totale dei crediti e dei debiti separatamente
      double crediti = transazioni.stream().filter(t -> t.getTipo().equals("credito")).map(t -> t.getImporto()).reduce(0.0, (a,b) -> a+b);
      double debiti = transazioni.stream().filter(t -> t.getTipo().equals("debito")).map(t -> t.getImporto()).reduce(0.0, (a,b) -> a+b);
      System.out.println("Totale crediti: " + crediti);
      System.out.println("Totale debiti: " + debiti);
```

Esercizio – Transazioni - Soluzione

```
        // Punto 3: Trovare la transazione con l'importo più alto in valore assoluto (crediti e debiti)
        Optional<Transazione> maxCredito = transazioni.stream().filter(t -> t.getTipo().equals("credito")).max(Comparator.comparingDouble(t -> t.getImporto()));
        Optional<Transazione> maxDebito = transazioni.stream().filter(t -> t.getTipo().equals("debito")).max(Comparator.comparingDouble(t -> t.getImporto()));
        System.out.println("Transazione massima in credito: " + maxCredito.orElse(null));
        System.out.println("Transazione massima in debito: " + maxDebito.orElse(null));


        // Punto 4: Ordinare le transazioni i nbase alla data in ordine cronologico
        List<Transazione> transazioni_ordinate = transazioni.stream().sorted((t1,t2) -> t1.getData().compareTo(t2.getData())).collect(Collectors.toList());
        System.out.println(transazioni_ordinate);
    }
}
```

Esercizio 7 - Prodotti

- Hai una lista di prodotti, ognuno rappresentato da una classe Prodotto con attributi nome, prezzo e categoria.
- Utilizza la Stream API per:

1. **Filtrare** i prodotti di una categoria specifica e con un prezzo superiore a una certa soglia.
2. **Trovare** il prodotto più caro di ogni categoria.
3. **Calcolare** il prezzo medio di tutti i prodotti.
4. **Creare** una mappa dove la chiave è la categoria e il valore è una lista dei nomi dei prodotti in quella categoria.

Esercizio 7 - Prodotti

```
public class Prodotto {
   private String nome, categoria;
   private double prezzo;

   public Prodotto(String nome, double prezzo, String categoria){
      this.nome = nome;
      this.prezzo = prezzo;
      this.categoria = categoria;
   }

   public String getNome(){
      return nome;
   }

   public double getPrezzo(){
      return prezzo;
   }

   public String getCategoria(){
      return categoria;
   }

   public String toString(){
      return nome + " , " + prezzo + " , " + categoria;
   }
}
```

Esercizio 7 - Prodotti - Soluzione

```
import java.util.*;
import java.util.stream.Collectors;
public class Esercizio7 {
   public static void main(String[] args){
      List<Prodotto> prodotti = Arrays.asList(
        new Prodotto("Tastiera", 50.0, "informatica"),
        new Prodotto("Pallone", 12.0, "sport"),
        new Prodotto("Quaderni", 5.0, "scuola"),
        new Prodotto("Mouse", 25.0, "informatica"),
        new Prodotto("Scarpe", 80.0, "abbigliamento")
      );
      // Punto 1: filtrare i prodotti di una categoria specifica e con un prezzo superiore ad una certa soglia
      List<Prodotto> prodottiFiltrati = prodotti.stream().filter(p -> p.getCategoria().equals("informatica") && p.getPrezzo()>30).collect(Collectors.toList());
      System.out.println(prodottiFiltrati);
      // Punto 2: trovare il prodotto più caro di ogni categoria
      Map<String, Optional<Prodotto>> prodottoPiuCaroPerCategoria = prodotti.stream().collect(Collectors.groupingBy(Prodotto::getCategoria,
Collectors.maxBy(Comparator.comparingDouble(Prodotto::getPrezzo))));
      prodottoPiuCaroPerCategoria.forEach((categoria, prodotto) -> System.out.println(categoria + ": " + prodotto.orElse(null)));
```

Esercizio 7 - Prodotti - Soluzione

```
        // Punto 3: calcolare il prezzo medio di tutti i prodotti
        double media = prodotti.stream().mapToDouble(Prodotto::getPrezzo).average().orElse(0.0);
        System.out.println("Prezzo medio: "+media);
        // Punto 4: creare una mappa dove la chiave è la ategoria e il valore è una lista dei nomi dei prodotti in quella categoria
        Map<String, List<String>> prodottiPerCategorie = prodotti.stream()
            .collect(Collectors.groupingBy(Prodotto::getCategoria, Collectors.mapping(Prodotto::getNome, Collectors.toList())));

        prodottiPerCategorie.forEach((categoria, nomi) ->
            System.out.println(categoria + ": " + nomi));
    }
}
```

Esercizio 8 - Libri

- Hai a disposizione una classe Book con i seguenti attributi:

```
public class Book {
    private String title;
    private String author;
    private int publicationYear;
    private double price;
    public Book(String title, String author, int publicationYear, double price) {
        this.title = title;
        this.author = author;
        this.publicationYear = publicationYear;
        this.price = price;
    }
    public String getTitle() {
        return title;
    }
    public String getAuthor() {
        return author;
    }
    public int getPublicationYear() {
        return publicationYear;
    }
    public double getPrice() {
        return price;
    }
    @Override
    public String toString() {
        return title + " by " + author + " (" + publicationYear + "), $" + price;
    }
}
```

Esercizio 8 - Libri

- Supponi di avere una lista di libri (List<Book> books) e di voler eseguire varie operazioni utilizzando la Stream API di Java. Scrivi il codice necessario per risolvere i seguenti punti:
- Filtro libri pubblicati dopo il 2010: Estrai una lista di libri pubblicati dopo il 2010.
- Ordinamento per prezzo: Ordina la lista di libri in ordine crescente di prezzo.
- Titoli di autori specifici: Crea una lista dei titoli dei libri scritti da un autore specifico (ad esempio, "J.K. Rowling").
- Media dei prezzi: Calcola la media dei prezzi di tutti i libri.
- Libro più costoso: Trova il libro con il prezzo più alto.
- Verifica prezzo massimo: Controlla se c'è almeno un libro che costa più di 30 euro.
- Stampa i libri ordinati per anno di pubblicazione in ordine decrescente.

Esercizio 8 - Libri - Soluzione

```
import java.util.*;
import java.util.stream.Collectors;
public class Esercizio8 {
   public static void main(String[] args){
      List<Book> books = Arrays.asList(
        new Book("La divina commedia", "Dante Alighieri", 1304, 40),
        new Book("I promessi sposi", "Alessandro Manzoni", 1827, 20),
        new Book("Big Brother", "George Orwell", 1984, 30),
        new Book("Infinite Jest", "David Wallace", 2011, 50)
      );
      // Punto 1: filtrare i libri pubblicati dopo il 2010
      List<Book> recentBooks = books.stream().filter(b -> b.getPublicationYear()>2010).collect(Collectors.toList());
      System.out.println(recentBooks);
      // Punto 2: ordinare la lista di libri in ordine crescente di prezzo
      List<Book> sortedBooks = books.stream().sorted(Comparator.comparingDouble(Book::getPrice)).collect(Collectors.toList());
      System.out.println(sortedBooks);
      // Punto 3: creare una lista dei titoli dei libri scritti da un autore specifico
      List<Book> authorBooks = books.stream().filter(b -> b.getAuthor().equals("Alessandro Manzoni")).collect(Collectors.toList());
      System.out.println(authorBooks);
```

Esercizio 8 - Libri - Soluzione

```
        // Punto 4: calcolare la media dei prezzi di tutti i libri
        double averageBooks = books.stream().mapToDouble(Book::getPrice).average().orElse(0.0);
        System.out.println("Prezzo medio: "+averageBooks);


        // Punto 5: trovare il libro con il prezzo più alto
        Optional<Book> maxBook = books.stream().max(Comparator.comparingDouble(b -> b.getPrice()));
        System.out.println(maxBook.orElse(null));


        // Punto 6: controllare se c'è almeno un libro che costa più di 30 euro
        boolean overBook = books.stream().anyMatch(b -> b.getPrice()>30);
        System.out.println(overBook);

        // Punto 7: stampare i libri ordinati per anno di pubblicazione in ordine decrescente
        List<Book> sortedBooks2 = books.stream().sorted(Comparator.comparingDouble(Book::getPublicationYear).reversed()).collect(Collectors.toList());
        System.out.println(sortedBooks2);
    }
}
```

Esercizio 9 - Analisi di una lista di persone

- Crea una classe Persona con i seguenti attributi:
  - nome (String)
  - età (int)
  - genere (enum Genere con valori MASCHIO e FEMMINA)
  - città (String)
- Crea una lista di oggetti Persona con almeno 10 persone, variando età, genere e città.

Esercizio 9 - Analisi di una lista di persone

Utilizza la Stream API per risolvere i seguenti problemi:

a) Filtrare le persone che hanno più di 30 anni.

b) Estrarre i nomi delle persone che vivono in una città specifica (es. "Milano").

c) Calcolare l'età media delle persone di sesso femminile.

d) Trovare la persona più giovane nella lista.

e) Raggruppare le persone per città e stampare il risultato.

f) Verificare se tutte le persone della lista sono maggiorenni.

g) Contare quante persone vivono in una città specifica.

Esercizio 9 - Analisi di una lista di persone - Soluzione

```
enum Genere {
   MASCHIO,
   FEMMINA }
public class Persona {
   private String nome, citta;
   private int eta;
   private Genere genere;
   public Persona(String nome, int eta, Genere genere, String citta){
      this.nome = nome;
      this.eta = eta;
      this.genere = genere;
      this.citta = citta;  }
   public String getNome() {
      return nome;
   }
   public int getEta() {
      return eta;
   }
   public Genere getGenere() {
      return genere;
   }
   public String getCitta() {
      return citta;
   }
   public String toString(){
      return nome + " " + eta + " " + genere + " " + citta;
   }
}
```

Esercizio 9 - Analisi di una lista di persone - Soluzione

```
public class Main {
    public static void main(String[] args) {
        List<Persona> persone = Arrays.asList(
            new Persona("Alice", 28, Genere.FEMMINA, "Milano"),
            new Persona("Marco", 35, Genere.MASCHIO, "Roma"),
            // altre persone
        );
        // a) Filtrare persone con età > 30
        List<Persona> maggiori30 = persone.stream()
            .filter(p -> p.getEtà() > 30)
            .collect(Collectors.toList());
        // b) Estrarre nomi delle persone che vivono a Milano
        List<String> nomiMilano = persone.stream()
            .filter(p -> p.getCittà().equals("Milano"))
            .map(Persona::getNome)
            .collect(Collectors.toList());
```

Esercizio 9 - Analisi di una lista di persone - Soluzione

```
        // c) Calcolare età media delle donne
        OptionalDouble etàMediaFemmine = persone.stream()
            .filter(p -> p.getGenere() == Genere.FEMMINA)
            .mapToInt(Persona::getEtà)
            .average();
        // d) Trovare la persona più giovane
        Optional<Persona> piùGiovane = persone.stream()
            .min(Comparator.comparingInt(Persona::getEtà));
        // e) Raggruppare per città
        Map<String, List<Persona>> personePerCittà = persone.stream()
            .collect(Collectors.groupingBy(Persona::getCittà));
        // f) Verificare se tutte le persone sono maggiorenni
        boolean tutteMaggiorenni = persone.stream()
            .map(p -> p.getEtà() >= 18)
            .reduce(True,(a,b) -> a && b);
        // g) Contare quante persone vivono a Roma
        long conteggioRoma = persone.stream()
            .filter(p -> p.getCittà().equals("Roma"))
            .count();
    }
}
```